

Artificial Intelligence and Knowledge Engineering

Reinforcement Learning

Halina Kwaśnicka, Maciej Piasecki

Choose yourself and new technologies



HUMAN CAPITAL
HUMAN – BEST INVESTMENT!



Wrocław University of Technology

EUROPEAN
SOCIAL FUND



Project co-financed from the EU European Social Fund



Reinforcement Learning paradigm^{*}

- Nature of learning: we learn by interacting with our environment
- When an newborn plays, waves its arms, or looks about, has no explicit teacher, but **does have a direct sensorimotor connection** to its environment
- Experiencing this connection produces a wealth of information about:
 - cause and effect,
 - the consequences of actions,
 - what to do in order to achieve goals.
- Throughout our lives, such interactions are undoubtedly a major source of knowledge about our environment and ourselves

Learning from interactions is a foundational idea underlying nearly all theories of learning and intelligence

***The lecture is based mainly on: Richard S. Sutton and Andrew G. Barto (2020)Reinforcement Learning: An Introduction. Second Edition, MIT Press, Cambridge.**

<http://incompleteideas.net/book/the-book-2nd.html>



Reinforcement Learning (RL) – an idea

- Reinforcement learning (RL) is
 - learning what to do
 - how to map situations to actions
 - so as to maximise a numerical reward signal
- The learner is not told which actions to take, as in most forms of machine learning, but instead must discover which actions yield the most reward by trying them
- In the most interesting cases, actions may affect not only the immediate reward, but also the next situation and all subsequent rewards
- These two characteristics – trial-and-error search and delayed reward – are the two most important distinguishing features of reinforcement learning



What makes reinforcement learning different from other machine learning paradigms?

- There is no supervisor, only a *reward* signal
- Feedback is delayed, not instantaneous
- Time really matters (sequential, **non i.i.d data!**)
 - “i.i.d.” means: independent and identically distributed variables
- Agent’s actions affect the subsequent data it receives



Learning agents

- Learning agent seeks to achieve a goal despite uncertainty about its environment
- The agent's action can influence the future state of the environment (e.g., the next location of the robot) – it affects the options and opportunities at a later time
- The correct choice should take into account the delayed consequences of actions, i.e., it requires foresight or planning
- The effects of actions cannot be fully predicted – the agent must monitor its environment frequently and react appropriately



An RL agent may include one or more of these components:

- **Policy**: agent's behaviour function:
state $\rightarrow$ action
- **Value function**: how good is each state
and/or action (prediction of future rewards)
- **Model**: agent's representation
of the environment, including states



RL Elements: a *policy*

- Basic elements are: a *policy*, a *reward function*, a *value function*, and, optionally, a *model* of the environment
 - **Policy** is the core of an RL agent in the sense that it alone is sufficient to determine behaviour
 - A **policy** – is a mapping from perceived *states* of the environment to *actions* to be taken in those states (a kind of set of stimulus-response rules or associations)
 - A **policy** may be a simple function or lookup table, or may involve extensive computation, such as a search process, or maybe stochastic
- deterministic: $a = \pi(s)$ or stochastic: $\pi(a|s) = P[A_t = a|S_t = s]$



RL Elements: a *reward function* and *reward*

- **Reward function** defines the goal in an RL problem
- It **maps** a perceived **state** (or state-action pair) to a single number – a **reward** (a scalar feedback signal)
- It defines what are good and bad events for a learning agent
- It must necessarily be unalterable by the agent
 - It is **a property of the environment**, not an agent
- A reward function may be stochastic
- A **reward** indicates the **intrinsic desirability** of a state
- RL's objective is the **maximisation of the total reward in the long run**



RL Elements: a *value function* (state value)

- In opposition to the *reward function*, a *value function* (i.e., *value of a state*) specifies what is good in the long run
- *Value function* is a prediction of future rewards
- Used to evaluate the goodness/badness of states
- and therefore to select between actions, e.g.:

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

- *State value* is the total amount of reward expected by an agent to accumulate over the future states, starting from the given one
- *State value* indicates the *long-term* desirability of states after taking into account the states that are likely to follow, and the rewards available in those states



A value function role

- We seek actions that bring about states of highest value, not highest reward, because these actions obtain the greatest amount of reward for us over the long run
- It is hard to determine values; rewards are given directly by the environment, but values **must be estimated and re-estimated from the sequences of observations** an agent makes over its entire lifetime
- In fact, the most important component of almost all RL algorithms is a method for efficiently estimating values

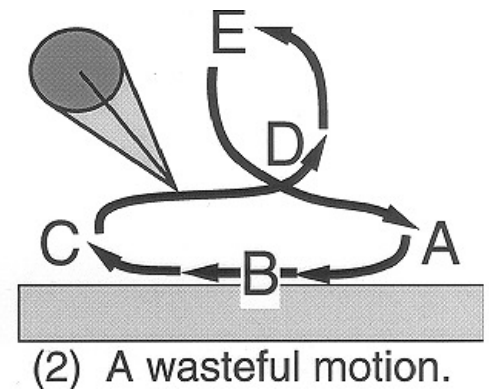
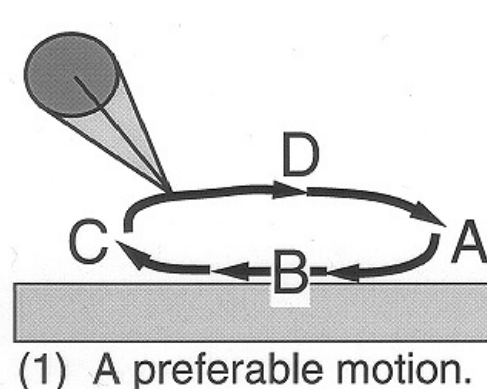
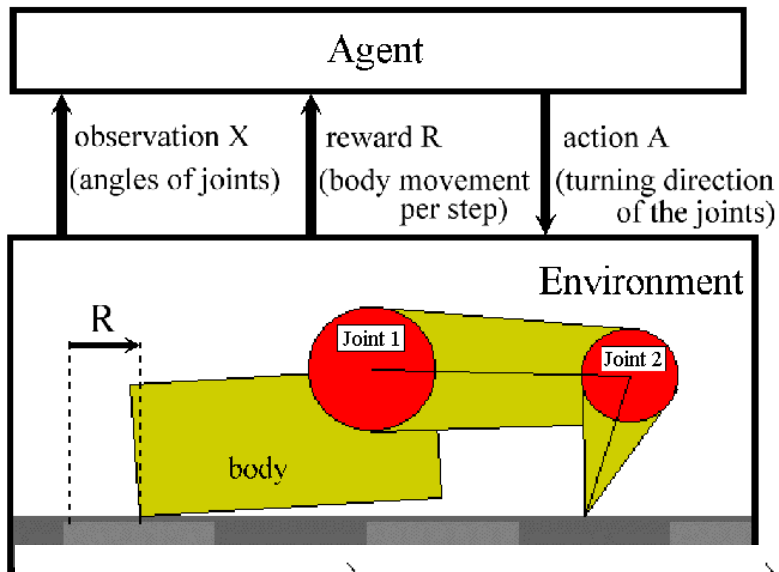


The RL framework

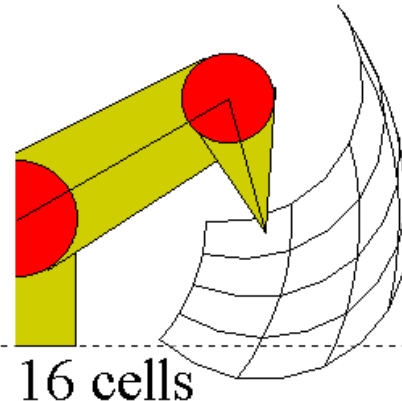
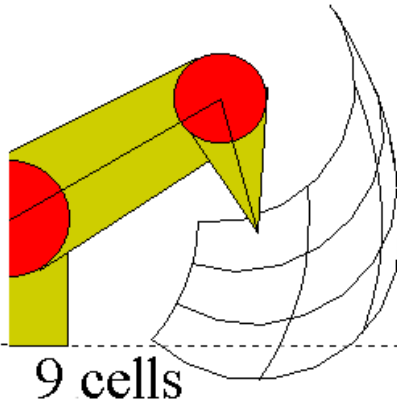
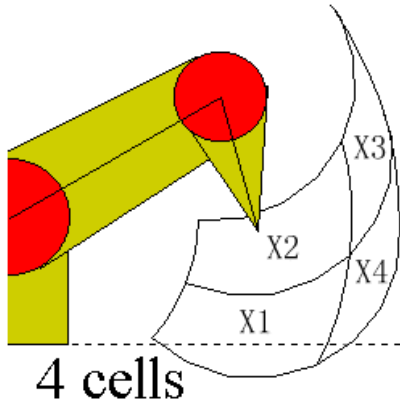
- At each time step, an agent implements a mapping from states to probabilities of selecting each possible action
- This mapping is called the agent's *policy* and is denoted Π_t , where $\Pi_t(s, a)$, is the probability of $a_t = a$ when $s_t = s$
- RL methods specify how an agent changes its policy as a result of its experience
- The agent's goal is to maximise the total amount of reward it receives over the long run
- The use of a reward signal to formalise the idea of a goal is one of the most distinctive features of RL



RL in Robotics

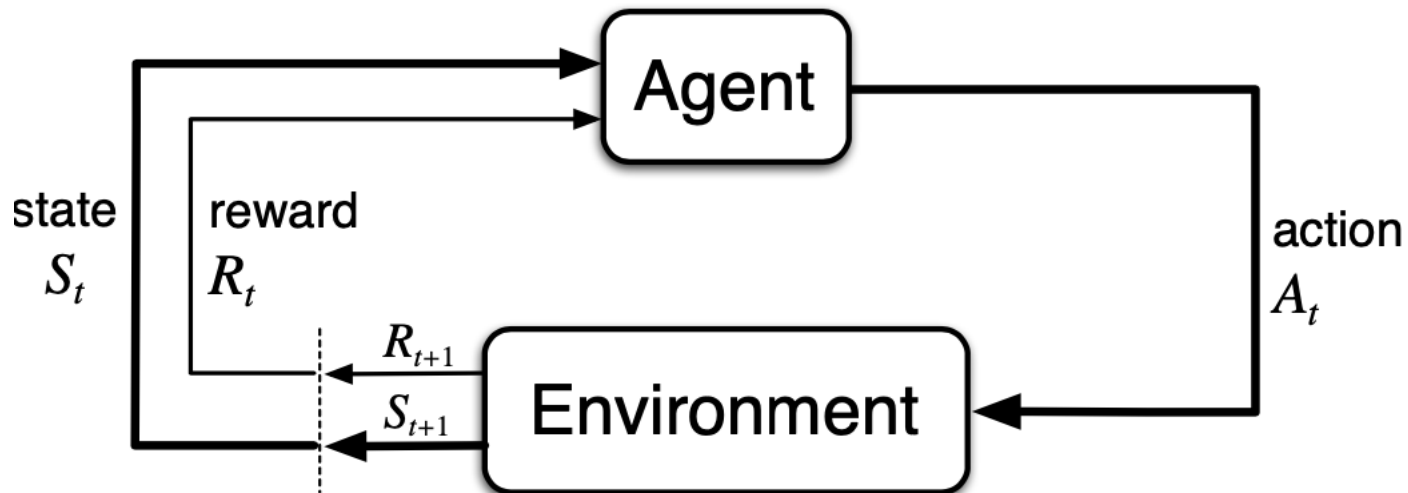


<http://sysplan.nams.kyushu-u.ac.jp/gen/papers/JavaDemoML97/robodemo.html>





RL: learning from interaction



- At each t , the agent receives some representation of an environment's *state*, $s_t \in S$, where S is a set of possible states,
- On that basis, the agent selects an *action*, $a_t \in A(s_t)$, where $A(s_t)$ is the set of actions available in state s_t
- The agent receives a numerical *reward*, $r_{t+1} \in R$, and finds itself in a new state s_{t+1}



Discounted return

- An agent tries to maximise the sum of discounted rewards it receives over the future, it chooses a_t to maximise the expected *discounted return*:

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

where γ is a parameter called the discount rate, and $0 \leq \gamma \leq 1$

- A reward received k time steps in the future is worth γ^{k-1} times what it would be worth if it were received immediately
- $\gamma < 1$ – the infinite sum has a finite value when $\{r_k\}$ is bounded
- $\gamma = 0$ – an agent maximises immediate rewards
- $\gamma = 1$ – an agent becomes more farsighted



Learning value function

- During the play the values of the ‘visited’ states are changed
- Making estimation of the probabilities of winning more accurate:
We “back up” the state’s value after each greedy move to the state before the move, i.e., the current value of the earlier state is adjusted to be closer to the value of the later state
- This can be done by moving the earlier state's value a fraction of the way toward the value of the later state



Temporal-difference learning – an idea (repetition)

- Let s denotes the state before the greedy move, and s' the state after the move, then the update to the estimated value of s , denoted $V(s)$, is:

$$V(s) \leftarrow V(s) + \alpha [V(s') - V(s)]$$

where α is a small positive fraction called the *step-size parameter*, which influences the rate of learning

- The above rule is an example of a *temporal-difference* learning method, so called because its changes are based on a difference, $V(s') - V(s)$, between estimates at two different times



RL Elements: Q-learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

(Sutton and Barto, 2020, pp. 131)

- It converges to the optimal policy for the model under conditions:
 - each state-action pair must be selected an infinite number of times in Step 1
 - and α must decrease appropriately over time
- During learning: maximal action a selected with probability $1-\varepsilon$, any with ε



An example: Tic-Tac-Toe

- Final states – states with reward:
winning, losing, a draw
- Let us consider draws and losses to
be equally bad for us:
how might we construct a player that
will find the imperfections in its
opponent's play and learn to
maximise its chances of winning?
- The classical „minimax" solution
from game theory is not correct here,
because it assumes a particular way
of playing by the opponent

X	O	O
O	X	X
		X

opponent's move

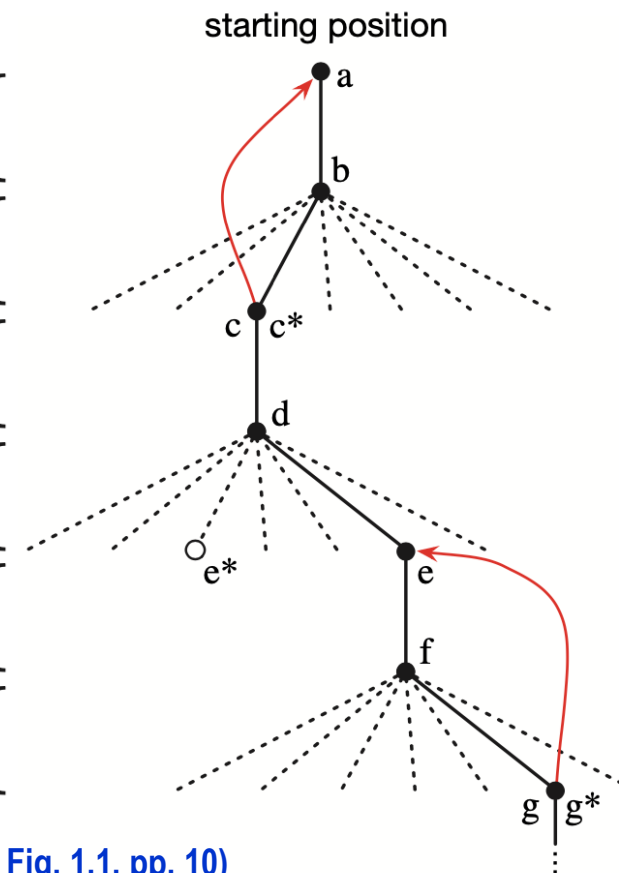
our move

opponent's move

our move

opponent's move

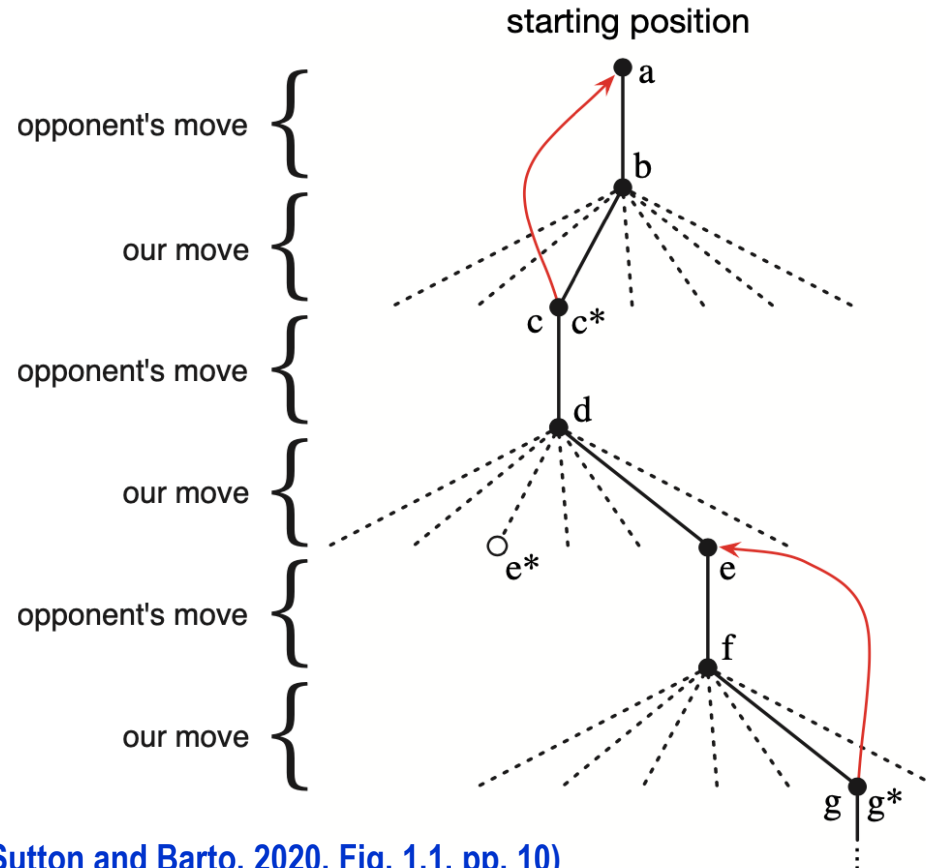
our move



(Sutton and Barto, 2020, Fig. 1.1, pp. 10)



- FIG.: A sequence of tic-tac-toe moves
- The solid lines represent the moves taken during a game
- The dashed lines represent moves that we (our RL player) considered but did not make.
- Our second move was an **exploratory move**, meaning that it was taken even though another sibling move, the one leading to e^* , was ranked higher



(Sutton and Barto, 2020, Fig. 1.1, pp. 10)

- Exploratory moves do not result in any learning, but each of our other moves does, causing updates as suggested by the curved arrow, in which estimated values are moved up the tree from later nodes to earlier, as detailed in the text



- While we are playing, we change the *values of the states* of the game
- We attempt to make them more accurate estimates of the probabilities of winning
- To do this, we “back up” the value of the state after each greedy move to the state before the move, as suggested by the arrows in Figure 1.1.
- More precisely, the current value of the earlier state is updated to be closer to the value of the later state
- This can be done by moving the earlier state's value a fraction of the way towards the value of the later state
- If we let s denote the state before the greedy move, and s' the state after the move, then the update to the estimated value of s , denoted $V(s)$, can be written as:

$$V(s) \leftarrow V(s) + \alpha [V(s') - V(s)]$$

where α is a small positive fraction called the *step-size parameter*, which influences the rate of learning

- This update rule is an example of a *temporal-difference learning method* – changes are based on a difference, $V(s') - V(s)$, between estimates at two different times



How to solve?

- Classical optimisation methods for sequential decision problems (e.g., *dynamic programming*), can compute an **optimal solution** for any opponent, but require as input a complete specification of that opponent, including the probabilities of each of his moves in each board state
- Let us assume that this information is not available
- Such information can be estimated from experience, in this case, by playing many games against the opponent
- The best one can do is first to learn a model of the opponent's behaviour, up to some level of confidence, and then apply dynamic programming to compute an optimal solution given the approximate opponent model
- This is not that different from some of the RL methods, see (Sutton and Barto, 2020, Fig. 1.1, pp. 10)



Summary: Tic-Tac-Toe – a potential solution

- **Dynamic programming** can compute an optimal solution for any opponent, but a complete specification of the opponent is needed
 - i.e., also probabilities of making particular moves by the opponent
- Assuming that such information is not available
- This information can be estimated on the basis of experience – a number of games can be played against the opponent
- Good way: to learn a model of the opponent's behaviour (with some level of confidence) and then use dynamic programming to find an optimal solution

Conclusion: It is a kind of reinforcement learning



Tic-tac-toe and RL with approximation of value function

- A table of numbers, one for each possible state of the game, should be set, where:
 - each number – the latest estimate of the probability of our winning from that state
 - the numbers – estimated states' values
 - the table – the learned value function
- If a state A has a higher value than state B, it means that state A is 'better' than B. i.e., the current estimate of the probability of our winning from A is higher than it is from B



Continuation:

let's assume we always play X

- For all states with three Xs in a row, the probability of winning is 1 (we won)
- For all states with three Os in a row, the probability of winning is 0 (we lost)
- The initial values we can set as 0.5, an equal chance of winning
- We play many games against the opponent
- While selecting the move, we examine the accessible states (from the current one) and their current values in the table
- Most of the time, we move *greedily* – the selected move leads to a state with the greatest value, that is, with the highest estimated probability of winning
- Occasionally, we select randomly from among the other moves
- Such moves are called *exploratory* moves because they cause us to experience states that we might otherwise never see



Some features of the method

- The method converges to an optimal policy for playing the game
- It is learning while interacting with the environment
- There is a clear goal
- The correct behaviour requires planning or foreseeing that takes into account the delayed effects of one's choices.
- It can achieve the effects of planning and looking ahead without using a model of the opponent and without conducting an explicit search over possible sequences of future states and actions



Examples

- <http://sysplan.nams.kyushu-u.ac.jp/gen/papers/JavaDemoML97/robodemo.html>
- <http://umichrl.pbworks.com/w/page/7597582/demos%20of%20reinforcement%20learning?revision=22473862>